



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Rosas Cañada Abraham

N° de Cuenta: 318307866

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 08/03/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

En esta práctica se implementó un Pyraminx (rompecabezas tipo Rubik en forma de tetraedro) utilizando OpenGL con GLEW, GLFW y GLM. Se utilizaron las clases provistas por el laboratorio (Mesh, Shader, Camera, Window). El resultado es un tetraedro regular negro como base sobre el cual se proyectan 9 triángulos planos de color por cada cara, obteniendo el aspecto del Pyraminx real con sus separaciones negras uniformes.

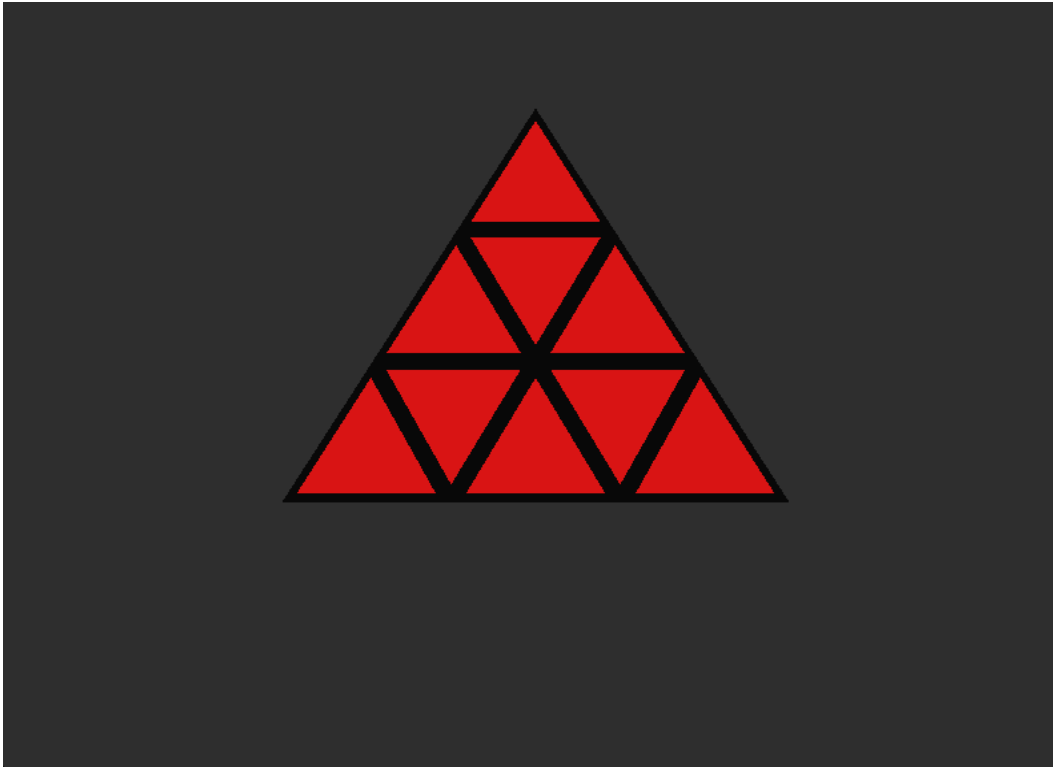


Figura 1. Cara frontal (roja) con sus 9 sub-triángulos y separaciones negras uniformes.

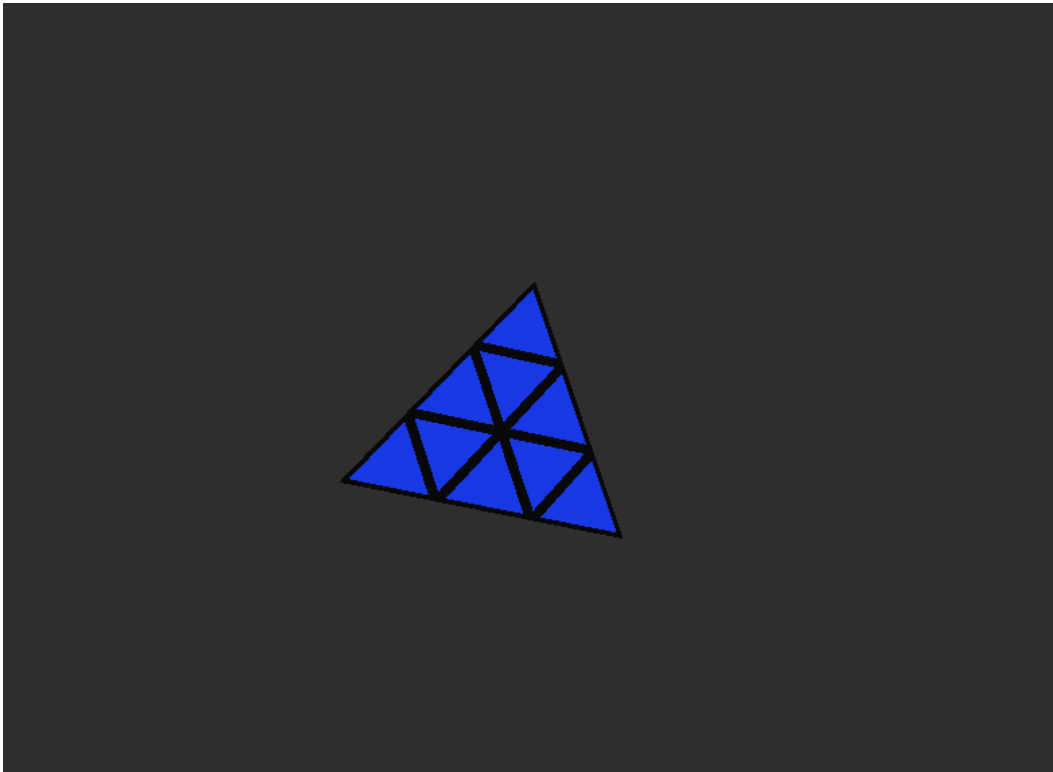


Figura 2. Cara lateral (azul) tras aplicar rotación con la cámara.

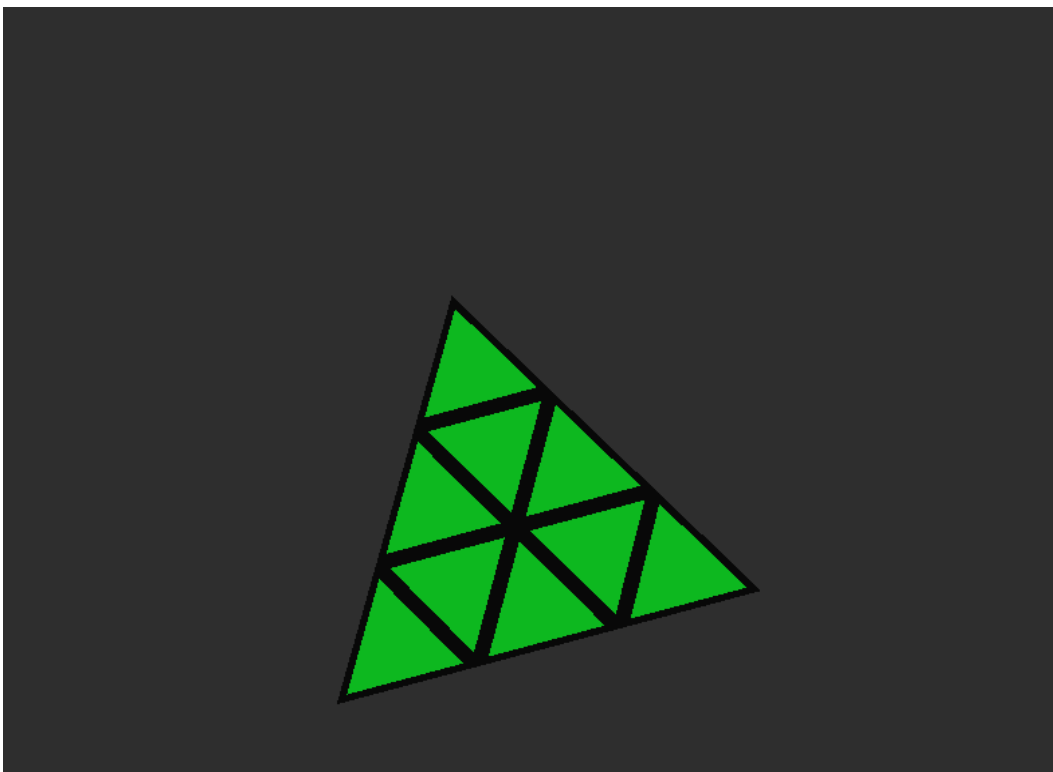


Figura 3. Cara verde con el patrón de 6 triángulos UP y 3 triángulos DOWN.

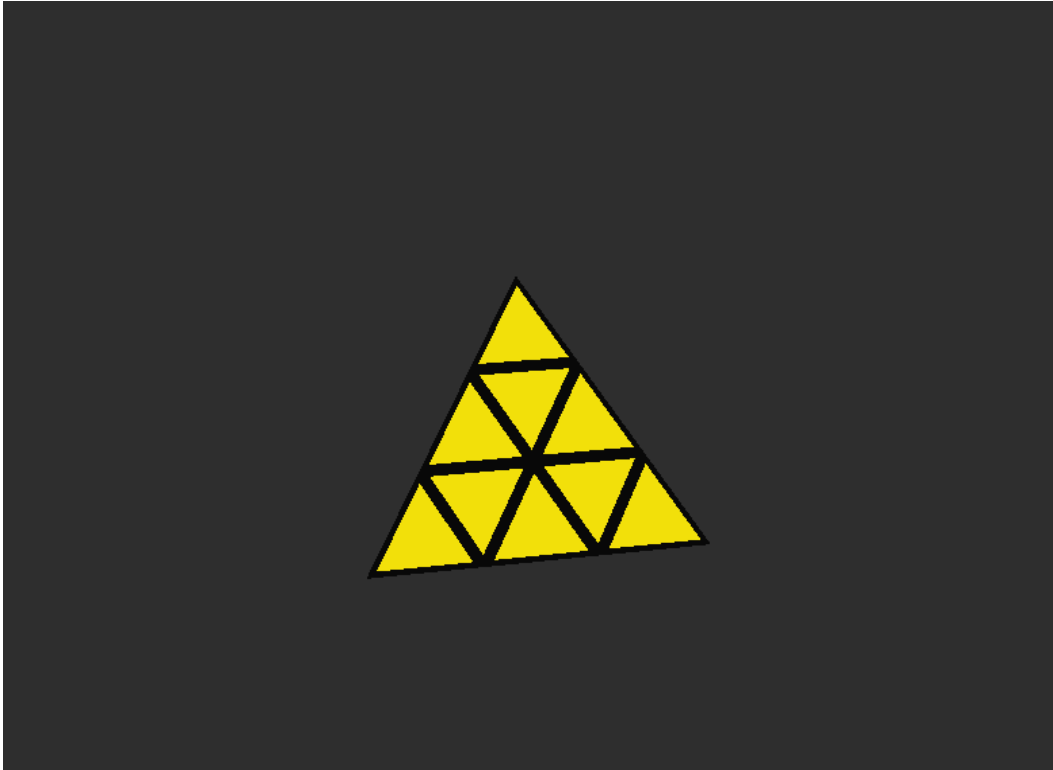


Figura 4. Cara base (amarilla) vista desde abajo.

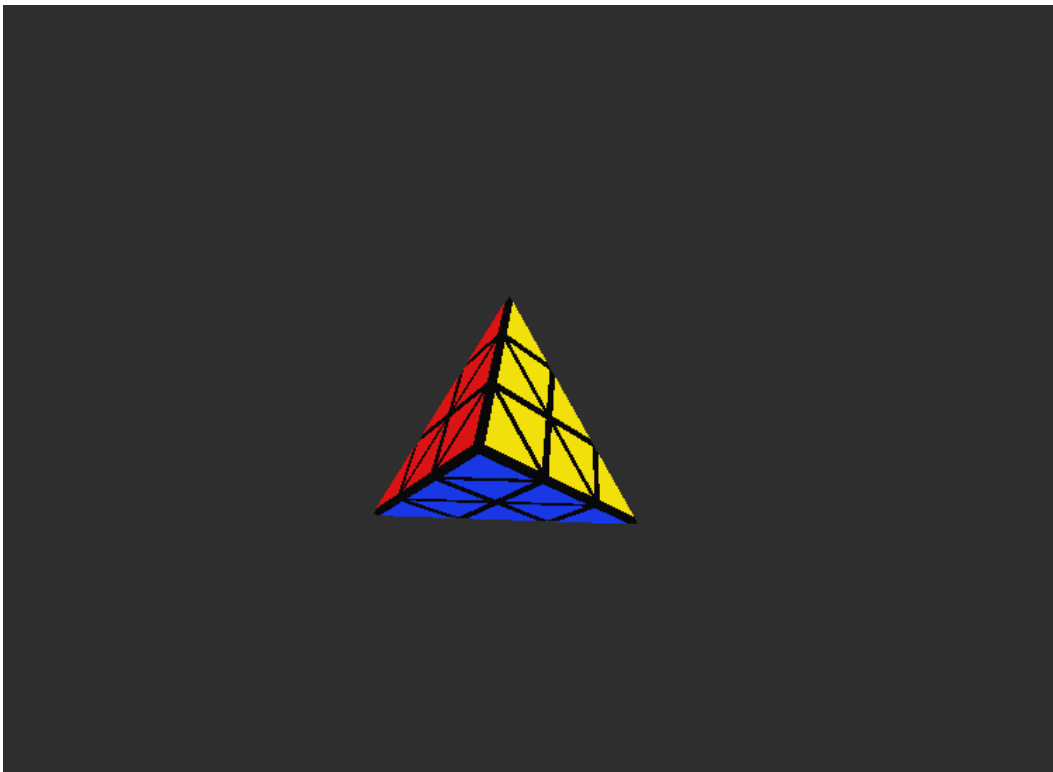


Figura 5. Vista completa con rotación (E/R/T), mostrando las 3 caras visibles simultáneamente.

1.1 Construcción del triángulo plano como sub-pieza

Se creó una malla de triángulo plano equilátero (meshList[1]) centrada en el origen en el plano Z=0. Al no tener profundidad se proyecta perfectamente sobre cada cara sin generar artefactos laterales. La malla del tetraedro regular negro (meshList[0]) sigue siendo la base estructural del Pyraminx.

```
// meshList[1]: Triangulo PLANO equilatero, centrado en origen, en plano Z=0
void CrearTrianguloPlano()
{
    const float h = 0.866025f; // sqrt(3)/2
    const float y0 = -0.288675f; // -h/3 (base)
    const float y2 = 0.577350f; // +2h/3 (punta)
    unsigned int indices[] = { 0,1,2, 0,2,1 }; // doble cara
    GLfloat vertices[] = {
        -0.5f, y0, 0.0f, 0.5f, y0, 0.0f, 0.0f, y2, 0.0f
    };
    Mesh* obj = new Mesh();
    obj->CreateMesh(vertices, indices, 9, 6);
    meshList.push_back(obj); // indice 1
}
```

1.2 Orientación y subdivisión de sub-piezas sobre cada cara

Cada cara del tetraedro se divide en 9 sub-triángulos mediante subdivisión paramétrica. La orientación de cada pieza se obtiene construyendo la matriz $rot=[t1|t2|n]$ a partir de los ejes tangentes reales de la cara. Los triángulos DOWN se rotan 180° en Z. El parámetro gapFactor controla el tamaño de cada pieza dejando visible el negro de fondo como borde.

```
// Ejes del plano de la cara
glm::vec3 t1 = glm::normalize(B - A);
glm::vec3 n = glm::normalize(glm::cross(B-A, C-A));
if (glm::dot(n, (A+B+C)/3.0f - centro) < 0.0f)
    n = -n;
glm::vec3 t2 = glm::normalize(glm::cross(n, t1));

// Matriz de orientacion: X->t1, Y->t2, Z->n
glm::mat4 rot = glm::mat4(1.0f);
rot[0]=glm::vec4(t1,0); rot[1]=glm::vec4(t2,0); rot[2]=glm::vec4(n,0);

// Subdivision:  $P(i,j) = A + (i/3)*(B-A) + (j/3)*(C-A)$ 
auto P = [&](int i, int j) { return A+(float(i)/3.0f)*(B-A)+(float(j)/3.0f)*(C-A); };

for (int j=0; j < 3; j++) {
    for (int i=0; i < 3-j; i++) {
        // UP: centroide + offset normal
        glm::vec3 cUp = (P(i,j)+P(i+1,j)+P(i,j+1))/3.0f + offN*n;
        meshList[1]->RenderMesh(); // triangulo plano de color
        if (i < 2-j) { // DOWN: rotar 180 en Z
            glm::vec3 cDn = (P(i+1,j)+P(i,j+1)+P(i+1,j+1))/3.0f + offN*n;
            meshList[1]->RenderMesh(); } } }
```

1.3 Rotación global del Pyraminx

Una única matriz tGlobal construida con las teclas E (eje X), R (eje Y) y T (eje Z) se aplica a todas las piezas, garantizando que el tetraedro negro y los 36 sub-triángulos roten en conjunto como un solo objeto.

```
// Rotacion global: teclas E(X), R(Y), T(Z)
glm::mat4 tGlobal = glm::mat4(1.0f);
tGlobal = glm::rotate(tGlobal, glm::radians(mainWindow.getrotax()), glm::vec3(1,0,0));
tGlobal = glm::rotate(tGlobal, glm::radians(mainWindow.getrotay()), glm::vec3(0,1,0));
tGlobal = glm::rotate(tGlobal, glm::radians(mainWindow.getrotaz()), glm::vec3(0,0,1));

// PASO 1: tetraedro negro (fondo - da bordes entre piezas)
mN = glm::scale(glm::mat4(1.0f), glm::vec3(ESC)); meshList[0]->RenderMesh();

// PASO 2: 9 triangulos planos de color por cada cara
for (int i=0; i < 4; i++) {
    DibujarCaraPyraminx(uniformModel, uniformColor, tGlobal,
        caras[i].A, caras[i].B, caras[i].C,
        glm::vec3(0.0f), caras[i].color, 0.82f, 0.015f);
}
```

2. Problemas encontrados y soluciones

Durante el desarrollo del modelo se presentaron varios problemas geométricos y de visualización que tuvieron que corregirse para obtener una representación adecuada del tetraedro. Inicialmente el tetraedro generado tenía lados desiguales, por lo que se recalcularon sus vértices para formar uno regular. También se detectó que las sub-piezas se veían como rombos y generaban líneas negras entre filas debido al uso de escala no uniforme, por lo que se reemplazaron por triángulos planos con escala uniforme. Además, la orientación de las piezas era incorrecta cuando se usaban rotaciones aproximadas, por lo que se ajustó su alineación respecto a cada cara. Finalmente, se corrigieron problemas de superposición visual y de posicionamiento para que las sub-piezas quedaran bien distribuidas sobre la superficie del tetraedro.

3. Conclusión

a) Complejidad y explicación del ejercicio

Esta práctica resultó ser de alta complejidad porque combinó álgebra lineal, geometría computacional y manejo de transformaciones en 3D de forma simultánea. Lo más importante que aprendí fue que para colocar objetos sobre superficies arbitrarias en el espacio 3D es necesario construir una base ortonormal (t_1 , t_2 , n) a partir de la geometría real de la superficie y usarla directamente como matriz de rotación. Esta técnica es fundamental en gráficos computacionales y es aplicable a muchas situaciones más allá de este ejercicio.

También aprendí que intentar simular una forma 2D aplastando un sólido 3D genera artefactos visuales difíciles de controlar. La solución correcta es siempre crear la geometría adecuada desde el inicio, en este caso un triángulo verdaderamente plano. Esto simplificó el problema eliminando la dimensión extra que causaba todos los errores visuales.

Otro aprendizaje importante fue el manejo del depth buffer y el z-fighting: cuando dos superficies están muy cerca, el GPU no puede determinar cuál está encima, generando artefactos visuales. La solución con un pequeño offset en la dirección de la normal es una técnica estándar en gráficos en tiempo real.

En cuanto a la complejidad del ejercicio, fue considerable: requirió corregir la geometría base del tetraedro, implementar la orientación por bases ortonormales, resolver el z-fighting y replantear la geometría de las sub-piezas. Cada problema fue un aprendizaje independiente que en conjunto resultó en una comprensión más profunda del pipeline de OpenGL y de la matemática detrás de las transformaciones 3D.

Bibliografía

- Angel, E., & Shreiner, D. (2015). Interactive computer graphics: A top-down approach with WebGL (7th ed.). Pearson Education.
- Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1996). Computer graphics: Principles and practice (2nd ed.). Addison-Wesley Professional.
- de Vries, J. (2020). LearnOpenGL: Learn modern OpenGL graphics programming in a step-by-step fashion. Kendall & Welling. <https://learnopengl.com>
- Shirley, P., & Marschner, S. (2009). Fundamentals of computer graphics (3rd ed.). A K Peters/CRC Press.
- OpenGL Mathematics (GLM). (2023). GLM documentation. <https://glm.g-truc.net>